

運用 Gemini 3 Flash 和 AlloyDB AI 建構 Autonomous Supply Chain

來源：<https://codelabs.developers.google.com/visual-commerce-gemini-3-alloydb?hl=zh-tw>

步驟數：13

使用 Gemini 3 Flash (程式碼執行) 建構「看得到」實體庫存的代理供應鏈系統、使用 AlloyDB AI (ScaNN)「記住」數百萬個零件，以及使用 A2A 通訊協定「交易」。

目錄

1. 總覽
2. 事前準備
3. 資料庫設定
4. 瞭解架構
5. 步驟 1：記憶體 (供應商代理程式)
6. 步驟 2：眼睛 (Vision 代理程式)
7. 步驟 3：握手 (A2A 代理商卡片)
8. 步驟 4：控制中心
9. 📖 額外內容：部署至 Cloud Run
10. 清除所用資源
11. 參考資料與延伸閱讀資源
12. 挑戰模式：提升代理能力
13. 結論

1. 總覽

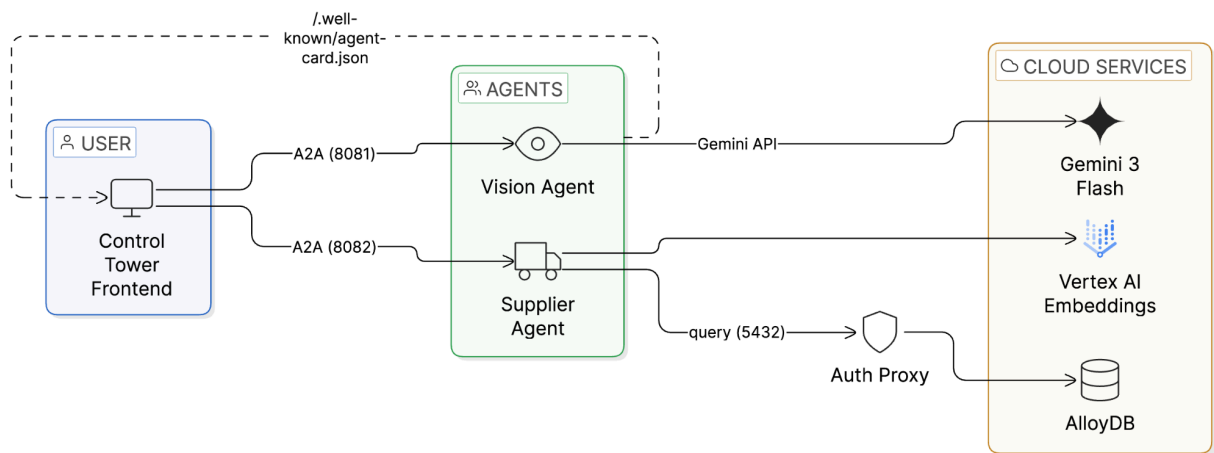
「會讀書的聊天機器人」時代即將結束。我們即將進入**智慧代理視覺**時代。

在本程式碼研究室中，我們將實作「**確定性 AI 工程**」，也就是建構不會猜測的 AI 系統。如果要求標準 AI 模型計算複雜圖片中的項目，模型通常會「產生幻覺」(猜測)。在供應鏈中，猜測很危險。如果 AI 猜測你有 12 個項目，但實際上你有 15 個，就會引發代價高昂的錯誤。

我們將運用 **Gemini 3 Flash** 中的全新「思考、行動、觀察」迴圈，建構**自主供應鏈代理**。不只是查看，而是調查。

確定性架構

我們會先從「盲目」和「失憶」系統開始。您將逐一「喚醒」其感官：



1. **眼睛 (Vision Agent)**：我們啟用 **Gemini 3 Flash** 的**程式碼執行**功能，模型不會預測權杖來猜測數字，而是會編寫 Python 程式碼 (OpenCV) 來確定像素數量。
2. **記憶體 (供應商代理程式)**：我們啟用 **AlloyDB AI** 和 **ScaNN** (可擴充的最近鄰)。這樣一來，代理程式就能在數毫秒內，從數百萬個選項中找出零件的確切供應商。
3. **握手 (A2A 通訊協定)**：我們使用標準化 `agent_card.json` 啟用**代理與代理**之間的通訊，讓 **Vision Agent** 自行向 **Supplier Agent** 下單。

建構項目

- 對攝影機畫面執行「視覺數學」的**視覺代理程式**。
- **供應商代理程式**，由 AlloyDB ScaNN 支援，可進行高速向量搜尋。
- **Control Tower** 前端，提供即時 WebSocket 更新，可將自主迴圈視覺化。

課程內容

- 如何設定 AlloyDB，並使用向量嵌入和 ScaNN 索引。
- 如何使用 Gemini API，透過 `gemini-3-flash-preview` 啟用 **Agentic Vision**。
- 瞭解如何在 AlloyDB 中使用 `<=>` (餘弦距離) 運算子實作向量搜尋。
- 如何使用 AlloyDB Python 連接器將代理程式連線至 AlloyDB。
- 如何使用 A2A 通訊協定動態探索代理。

需求條件

- [Chrome](#) 或 [Firefox](#) 瀏覽器
 - 已啟用計費功能的 Google Cloud 專案。
 - Vision Agent 適用的 Gemini API 金鑰 ([Google AI Studio](#) 提供免費方案)。
-

步驟 2 · 約 0 分鐘

2. 事前準備

info

從做中學 免付費

立即取得 Google Cloud 抵免額，並完成這個實驗室。無須提供信用卡或付款方式。

啟用

建立專案

1. 在 [Google Cloud 控制台](#) 的專案選取器頁面中，選取或建立 Google Cloud [專案](#)。
2. 確認 Cloud 專案已啟用計費功能。瞭解如何[檢查專案是否已啟用計費功能](#)。
3. 您將使用 [Cloud Shell](#)，這是 Google Cloud 中執行的指令列環境。點選 Google Cloud 控制台頂端的「啟用 Cloud Shell」。



4. 連至 Cloud Shell 後，請使用下列指令確認驗證已完成，專案也已設為獲派的專案 ID：

```
gcloud auth list
```

設定資料庫 [AlloyDB]

首先，請先佈建資料庫，這需要約 15 分鐘，因此我們先從這裡開始。

1. 點選下方按鈕，在 Cloud Shell 中開啟 AlloyDB 設定工具：
2. 執行設定：

```
Sh run.sh
```

3. 使用「網頁預覽」(眼睛圖示  → 「透過以下通訊埠預覽：8080」) 開啟設定 UI。
4. 輸入專案 ID、選取區域 (例如 us-central1)，然後建立資料庫密碼。

⚠ 請儲存這個密碼，設定指令碼要求時會用到。

5. 按一下「Start Deployment」(開始部署)，然後等待叢集佈建完成 (約 15 分鐘)。

取得程式碼

在 AlloyDB 佈建期間 (或完成後)，在 Cloud Shell 中開啟 Codelab 存放區：

⚠ 重要事項：點選按鈕後，系統會顯示安全性對話方塊。勾選「信任存放區」方塊，然後按一下「確認」。

或者，手動複製：

```
git clone https://github.com/MohitBhimrajka/visual-commerce-gemini-3-alloydb.git

cd visual-commerce-gemini-3-alloydb
```

設定專案

在這個 Cloud Shell 終端機中，確認已設定專案：

```
gcloud config set project <YOUR_PROJECT_ID>
```

在 AlloyDB 上啟用公開 IP

AlloyDB 佈建完成後，請啟用公開 IP，讓 Python 連接器可以從 Cloud Shell 連線：

1. 前往 [AlloyDB 控制台](#)
2. 依序點選叢集和主要執行個體
3. 按一下「編輯」
4. 捲動至「公開 IP 連線」，然後勾選「啟用公開 IP」
5. 按一下「更新執行個體」

💡 注意：AlloyDB Python 連接器會處理驗證和加密作業，因此您不需要新增任何已授權的外部網路。

授予 Vertex AI 權限

AlloyDB 服務帳戶需要 Vertex AI 存取權，才能生成嵌入。在同一個 Cloud Shell 視窗中執行下列指令：

```
PROJECT_ID=$(gcloud config get-value project)

gcloud projects add-iam-policy-binding $PROJECT_ID \
  --member="serviceAccount:service-$(gcloud projects describe $PROJECT_ID --
format="value(projectNumber)")@gcp-sa-alloydb.iam.gserviceaccount.com" \
  --role="roles/aiplatform.user"
```

執行設定指令碼

現在執行設定指令碼，系統會自動偵測 AlloyDB 執行個體：

```
sh setup.sh
```

指令碼功能：

- 驗證 gcloud CLI、驗證、專案和 Python 3
- 檢查並啟用必要 API (AlloyDB、Vertex AI、Compute、Service Networking)
- 系統會提示您輸入 Gemini API 金鑰
- 自動偵測 AlloyDB 執行個體，並擷取區域、叢集和執行個體名稱
- 要求提供資料庫密碼
- 產生 .env 設定檔

- 安裝 Python 依附元件

 疑難排解提示：

- 控制台中的專案有誤嗎？檢查控制台左上方的專案下拉式選單。
 - 無法建立 AlloyDB 執行個體？您必須先啟用計費功能，AlloyDB 是高效能引擎，因此必須提供這項資訊。
 - 啟用 API 後顯示「未啟用」？請等待 60 秒，因為傳播延遲是實際存在的。
 - 區域配額問題？如果 us-central1 失敗，請嘗試 us-east1 或 us-west1。
-

3. 資料庫設定

注意：如果您已完成完整的 Cloud Shell 教學課程，這些步驟都會自動完成。本節說明幕後發生的情況。

我們應用程式的核心是 **AlloyDB for PostgreSQL**。我們將運用其強大的向量功能和 ScaNN 索引，啟用近乎即時的語意搜尋，讓服務專員在毫秒內從數千筆記錄中找出相符的庫存。

在本節中，您將佈建結構定義、植入資料，並產生嵌入內容，所有作業都透過 AlloyDB Studio 進行。

連線至 AlloyDB Studio

1. 前往 [AlloyDB 控制台](#) 中的 AlloyDB 執行個體
2. 按一下左側導覽列中的「AlloyDB Studio」
3. 使用下列方式驗證：
4. 使用者名稱：postgres
5. 資料庫：postgres
6. 密碼：您在建立叢集時設定的密碼

啟用擴充功能

AlloyDB 提供向量和 AI 的內建擴充功能。在 AlloyDB Studio 中執行下列 SQL：

```
CREATE EXTENSION IF NOT EXISTS google_ml_integration CASCADE;  
CREATE EXTENSION IF NOT EXISTS vector;  
CREATE EXTENSION IF NOT EXISTS alloydb_scann CASCADE;
```

- **google_ml_integration**：啟用 ai.embedding() 函式，直接從 SQL 呼叫 Vertex AI。
- **向量**：儲存及查詢 768 維度的向量嵌入。
- **alloydb_scann**：啟用 Google 的 ScaNN 索引，進行超快速向量搜尋。

建立商品目錄資料表

```
DROP TABLE IF EXISTS inventory;  
  
CREATE TABLE inventory (  
    id SERIAL PRIMARY KEY,  
    part_name TEXT NOT NULL,  
    supplier_name TEXT NOT NULL,  
    description TEXT,  
    stock_level INT DEFAULT 0,  
    part_embedding vector(768)  
);
```

part_embedding 資料欄會儲存 **text-embedding-005** 的 **768** 維度向量。
這項技術是語意搜尋的基礎。

插入範例資料

插入 20 個倉庫商品目錄項目：

```
INSERT INTO inventory (part_name, supplier_name, description, stock_level) VALUES
('Cardboard Shipping Box Large', 'Packaging Solutions Inc', 'Heavy-duty corrugated cardboard shipping container, 24x18x12 inches', 250),
('Warehouse Storage Container', 'Industrial Supply Co', 'Stackable plastic storage bin with snap-lock lid, blue', 180),
('Product Shipping Boxes', 'Acme Packaging', 'Medium corrugated boxes for warehouse storage, 18x14x10 inches', 320),
('Industrial Widget X-9', 'Acme Corp', 'Heavy-duty industrial coupling for pneumatic systems', 50),
('Precision Bolt M4', 'Global Fasteners Inc', 'Stainless steel M4 allen bolt, 20mm length, grade A2-70', 200),
('Hexagonal Nut M6', 'Metro Supply Co', 'Galvanized steel hex nut M6, DIN 934 standard', 150),
('Phillips Head Screw 3x20', 'Acme Corp', 'Zinc-plated Phillips head wood screw, 3mm x 20mm', 500),
('Wooden Dowel 10mm', 'Craft Materials Ltd', 'Hardwood birch dowel rod, 10mm diameter x 300mm length', 80),
('Rubber Gasket Small', 'SealTech Industries', 'Buna-N rubber gasket, 25mm OD x 15mm ID, oil resistant', 120),
('Spring Tension 5kg', 'Mechanical Parts Co', 'Stainless steel compression spring, 5kg load capacity', 60),
('Bearing 6204', 'Bearings Direct', 'Deep groove ball bearing 6204-2RS, 20x47x14mm sealed', 45),
('Warehouse Shelf Boxes', 'Storage Systems Ltd', 'Standardized warehouse inventory boxes, corrugated, bulk pack', 400),
('Inventory Container Units', 'Supply Chain Pros', 'Modular stackable storage units for warehouse racking', 95),
('Aluminum Extrusion Bar', 'MetalWorks International', 'T-slot aluminum extrusion 20x20mm profile, 1 meter length', 110),
('Cable Tie Pack 200mm', 'ElectroParts Depot', 'Nylon cable ties, 200mm x 4.8mm, UV resistant black, pack of 100', 600),
('Hydraulic Hose 1/2 inch', 'FluidPower Systems', 'High-pressure hydraulic hose, 1/2 inch ID, 3000 PSI rated', 35),
('Safety Goggles Clear', 'WorkSafe Equipment Co', 'ANSI Z87.1 rated clear safety goggles, anti-fog coating', 275),
('Packing Tape Industrial', 'Packaging Solutions Inc', 'Heavy-duty polypropylene packing tape, 48mm x 100m, clear', 450),
('Stainless Steel Sheet 1mm', 'MetalWorks International', '304 stainless steel sheet, 1mm thickness, 300x300mm', 70),
('Silicone Sealant Tube', 'SealTech Industries', 'Industrial-grade RTV silicone sealant, 300ml cartridge, grey', 190);
```

授予嵌入權限

```
GRANT EXECUTE ON FUNCTION embedding TO postgres;
```

生成向量嵌入

這會使用 AlloyDB 的內建 `ai.embedding()` 函式，直接從 SQL 呼叫 Vertex AI 的 `text-embedding-005` 模型，不需要 Python 程式碼：

```
UPDATE inventory
SET part_embedding = ai.embedding(
    'text-embedding-005',
    part_name || ' . ' || description
)::vector
WHERE part_embedding IS NULL;
```

這會產生 768 維度的向量，擷取每個零件名稱和說明的語意。ScaNN 索引會使用這些資料，以極快的速度執行相似度搜尋。**[這項作業約需 3 至 5 分鐘]**

建立 ScaNN 索引

```
SET scann.allow_blocked_operations = true;

CREATE INDEX IF NOT EXISTS idx_inventory_scann
ON inventory USING scann (part_embedding cosine)
WITH (num_leaves=5, quantizer='sq8');
```

為什麼選擇 ScaNN？標準向量搜尋 (HNSW) 需要大量記憶體。當目錄成長到 1 億個零件時，索引會變得太大，無法存放在 RAM 中。**ScaNN (可擴充的最鄰近項目)** 使用向量量化壓縮向量，讓索引符合 CPU 的 L2 快取。因此，篩選查詢的速度最多比標準 HNSW 快 **10 倍**。

確認一切正常運作

```
SELECT part_name, supplier_name, stock_level,
       (part_embedding IS NOT NULL) as has_embedding
FROM inventory
ORDER BY id;
```

您應該會看到 20 個資料列，且所有資料列的 `has_embedding = true`。

疑難排解提示：

- 「找不到擴充功能」？執行個體可能仍在更新中。請稍候片刻，然後再試一次。
- 嵌入時顯示「權限遭拒」？IAM 傳播最多需要 60 秒。重新執行 GRANT 和 UPDATE 陳述式。
- 向量維度不符？該資料表使用 `vector(768)`，請改用 `text-embedding-005`。其他機型的尺寸不同。
- 嵌入傳回 **NULL**？請再次確認 AlloyDB 服務帳戶已獲授 Vertex AI 使用者 IAM 角色 (請參閱上一節)。

步驟 4 · 約 0 分鐘

4. 瞭解架構

在變更程式碼之前，請先瞭解系統的建構方式。架構採用漸進式「喚醒」模式：

代理程式堆疊

Vision Agent (agents/vision-agent/)

- agent.py：Gemini 3 Flash 的核心邏輯。將圖片傳送至已啟用程式碼執行的模型，以便撰寫 Python (OpenCV) 程式碼，以確定方式計算項目。
- agent_executor.py：將 A2A 通訊協定要求橋接至代理邏輯。
- main.py：Uvicorn A2A 伺服器，用於提供 /.well-known/agent-card.json 並處理要求。

供應商代理 (agents/supplier-agent/)

- inventory.py：透過 [AlloyDB Python 連接器](#) 連線至 AlloyDB (不需要 Auth Proxy)。包含執行 ScaNN 向量搜尋的 find_supplier() 函式。
- agent_executor.py：將 A2A 通訊協定橋接至目錄搜尋邏輯。
- main.py：Uvicorn A2A 伺服器，包含代理程式資訊卡和健康狀態端點。

Control Tower (frontend/)

- app.py：FastAPI + WebSocket 伺服器，可透過 A2A 探索代理程式、協調「影像 → 搜尋 → 訂購」管道，並將即時更新串流至瀏覽器。

A2A 流程

- Control Tower 會從每個代理讀取 /.well-known/agent-card.json
- 探索功能 (技能、端點) - 無需硬式編碼網址
- 將圖片傳送至 Vision Agent → 取得商品數量 + 說明
- 將說明做為嵌入查詢傳送至供應商代理 → 取得部分相符結果
- 自主下單

AlloyDB 連線

供應商代理程式會使用 **AlloyDB Python 連接器**，而非傳統的 Auth Proxy：

```
from google.cloud.alloydbconnector import Connector

connector = Connector()
conn = connector.connect(
    inst_uri,          # Full instance URI
    "pg8000",          # Driver
    user="postgres",
    password=DB_PASS,
    ip_type="PUBLIC",  # Cloud Shell uses Public IP
)
```

這會自動處理 IAM 驗證、SSL/TLS 和連線路由。日後部署至 Cloud Run 時，只要將 `ip_type` 變更為「PRIVATE」，即可存取虛擬私有雲。

5. 步驟 1：記憶體 (供應商代理程式)

注意：如果您已完成 Cloud Shell 教學課程，就已進行這項程式碼變更。本節將說明這項功能的重要性及運作方式。

供應商代理程式會使用 **AlloyDB ScaNN** 記憶數百萬個零件。目前，這項功能會隨附預留位置查詢，也就是會傳回找到的第一列，無論您搜尋的內容為何。

The Audit：The Amnesiac

如果現在查詢供應商代理程式，系統會傳回隨機結果。沒有相似度概念。讓我們一起解決這個問題！

啟動供應商代理程式

A2A 伺服器 (main.py) 會委派給 agent_executor.py，後者會將通訊協定橋接至 [inventory.py](#) 中的商業邏輯。

 **疑難排解提示：**如果收到「地址已在使用中」錯誤訊息，請終止現有程序，然後再試一次：

```
killall -f uvicorn #Kill all uvicorn processes
```

步驟 1：前往代理程式目錄

```
cd agents/supplier-agent
```

步驟 2：安裝依附元件

```
pip install -r requirements.txt
```

步驟 3：啟動代理程式伺服器

```
uvicorn main:app --host 0.0.0.0 --port 8082 > /dev/null 2>&1 &
```

> /dev/null 2>&1 & 會在背景執行伺服器並抑制輸出內容，以免中斷終端機。

步驟 4：確認代理程式正在執行 (啟動後等待 2 到 3 秒)

```
curl http://localhost:8082/.well-known/agent-card.json
```

預期輸出內容：含有代理程式設定的 JSON (應會傳回，且沒有錯誤)

修正方式：實作 `<=>` 運算子

開啟 agents/supplier-agent/inventory.py，並在 **60 到 70** 行附近找到 find_supplier() 函式。您會看到預留位置：

```
# TODO: Replace this placeholder query with ScaNN vector search

sql = "SELECT part_name, supplier_name FROM inventory LIMIT 1;"
cursor.execute(sql)
```

將這兩行程式碼替換為：

```
sql = """
SELECT part_name, supplier_name,
       part_embedding <=> %s::vector as distance
FROM inventory
ORDER BY part_embedding <=> %s::vector
LIMIT 1;
"""

cursor.execute(sql, (embedding_str, embedding_str))
```

用途：

- `<=>` 是 PostgreSQL 中的餘弦距離運算子
- `ORDER BY part_embedding <=> %s::vector` 會找出最接近的相符項目 (距離最短 = 語意最接近)
- `%s::vector` 會將嵌入陣列轉換為 PostgreSQL 的向量型別
- ScaNN 索引會自動加快這項查詢的速度！

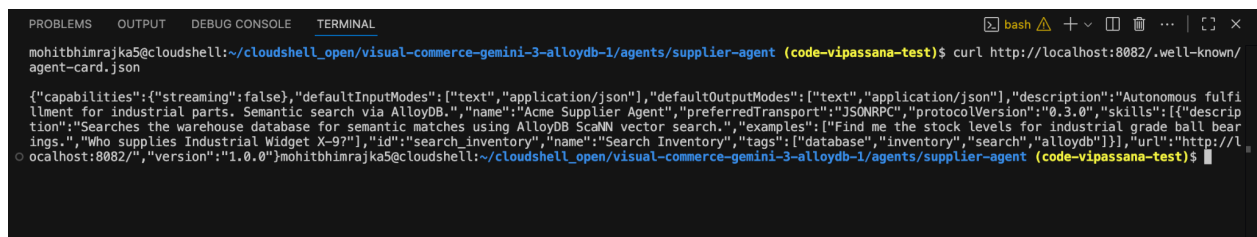
步驟 4：儲存檔案 (Ctrl+S 或 Cmd+S)

現在，代理程式會使用語意搜尋，而不是回傳隨機結果！

驗證

測試 A2A 探索功能和目錄：

```
curl http://localhost:8082/.well-known/agent-card.json
```



```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL
mohitbhimrajka5@cloudshell:~/cloudshell_open/visual-commerce-gemini-3-alloydb-1/agents/supplier-agent (code-vipassana-test)$ curl http://localhost:8082/.well-known/agent-card.json

{"capabilities":{"streaming":false,"defaultInputModes":["text","application/json"],"defaultOutputModes":["text","application/json"],"description":"Autonomous fulfillment for industrial parts. Semantic search via AlloyDB.", "name":"Acme Supplier Agent","preferredTransport":"JSONRPC","protocolVersion":"0.3.0","skills":[{"description":"Searches the warehouse database for semantic matches using AlloyDB ScaNN vector search.", "examples":["Find me the stock levels for industrial grade ball bearings.", "Who supplies Industrial Widget X-97?"]}, {"id":"search_inventory", "name":"Search Inventory", "tags":["database", "inventory", "search", "alloydb"]}]}}, {"url":"http://localhost:8082/", "version":"1.0.0"}mohitbhimrajka5@cloudshell:~/cloudshell_open/visual-commerce-gemini-3-alloydb-1/agents/supplier-agent (code-vipassana-test)$
```

```
python3 -c "  
from inventory import find_supplier  
import json  
vec = [0.1]*768  
r = find_supplier(vec)  
if r:  
    result = {'part': r[0], 'supplier': r[1]}  
    if len(r) > 2:  
        result['distance'] = float(r[2]) if r[2] else None  
    print(json.dumps(result))  
else:  
    print('No result found')  
"
```

預期行為：agent-card.json 會傳回代理程式資訊卡。Python 片段會從植入的資料傳回零件和供應商。

常見錯誤：

- 「找不到你所指定的檔案或目錄」：你不在存放區根目錄中。執行 `cd` 前往主目錄，然後執行 `cd visual-commerce-gemini-3-alloydb`，或使用 `pwd` 找出目前所在位置。
- 通訊埠 8082 正在使用中：終止現有程序：`lsof -ti:8082 | xargs kill -9`
- **ModuleNotFoundError**：安裝依附元件：`pip install -r requirements.txt`
- 資料庫連線失敗：檢查 `.env` 檔案，填入 `ALLOYDB_REGION`、`ALLOYDB_CLUSTER` 和 `ALLOYDB_INSTANCE`
- 連線遭拒：請確認 AlloyDB 執行個體已啟用公開 IP
- 驗證指令碼失敗：請確認種子已在步驟 4 中順利完成

6. 步驟 2：眼睛 (Vision 代理程式)

注意：如果您已完成 Cloud Shell 教學課程，代表您已啟用程式碼執行功能。本節將說明架構，以及這種做法為何能消除幻覺。

資料庫可供存取，現在就使用 **Gemini 3 Flash** 喚醒雙眼。Vision Agent 會透過程式碼執行作業執行「視覺數學」運算。

稽核：幻覺

如果詢問標準多模態模型「這張雜亂的圖片中有幾個盒子？」，模型會將圖片視為靜態快照並進行猜測。

- 模型：「我看到約 12 個方塊。」
- 實際情況：有 15 個盒子。
- 結果：供應鏈故障。

解決方法：喚醒思考、行動、觀察循環

我們啟用「程式碼執行」和「思考設定」，讓模型編寫 Python (OpenCV) 程式碼，以確定方式計算。

1. 開啟 `agents/vision-agent/agent.py`。
2. 找到 `GenerateContentConfig` 區段 (約在第 68 到 78 行)。
3. 取消註解 `thinking_config=types.ThinkingConfig(...)` 區塊和 `tools=[types.Tool(code_execution=...)]`。
4. 用戶端已設定為使用環境中的 `GEMINI_API_KEY`。

提示：`thinking_level="MINIMAL"` 可兼顧速度和準確度。有效值：「MINIMAL」、「LOW」、「MEDIUM」、「HIGH」。需要 `google-genai>=1.56.0`。

架構注意事項：Vision Agent 使用 Gemini API (API 金鑰)，Supplier Agent 則使用 Vertex AI 嵌入 (GCP 憑證)。這種混合式做法可簡化視覺元件，同時維持企業級資料庫功能。

檔案：`agents/vision-agent/agent.py`

```
config = types.GenerateContentConfig(
    temperature=0,
    # CODELAB STEP 1: Uncomment to enable reasoning
    thinking_config=types.ThinkingConfig(
        thinking_level="MINIMAL", # Valid: "MINIMAL", "LOW", "MEDIUM", "HIGH"
        include_thoughts=False    # Set to True for debugging
    ),
    # CODELAB STEP 2: Uncomment to enable code execution
    tools=[types.Tool(code_execution=types.ToolCodeExecution)]
)
```

為何 `thinking_level="MINIMAL"`？

對於這項任務 (透過執行程式碼計算項目)，「MINIMAL」提供的推論足以規劃指令碼並驗證計數。使用「高」設定會增加 2 到 3 倍的延遲時間，但不會提高確定性工作的準確度。**成本效益最佳化：**根據工作複雜度調整推論深度。

成本效益最佳化是生產 AI 工程的重要技能：根據工作複雜度調整推論深度。

啟動 Vision Agent

📁 路徑檢查：如果仍在 `agents/supplier-agent/` 中，請先使用 `cd ../..` 返回存放區根目錄。

步驟 1：前往 Vision 代理程式目錄

```
cd agents/vision-agent
```

步驟 2：安裝依附元件

```
pip install -r requirements.txt
```

步驟 3：啟動 Vision 代理程式伺服器

```
uvicorn main:app --host 0.0.0.0 --port 8081 > /dev/null 2>&1 &
```

`> /dev/null 2>&1 &` 會在背景執行伺服器並抑制輸出內容，以免中斷終端機。

驗證

測試 A2A 探索功能：

```
curl http://localhost:8081/.well-known/agent-card.json
```

預期行為：JSON，內含代理程式名稱和技能。您將在步驟 8 中，透過 Control Tower UI 測試實際的影像計數。

```
mohitbhimrajka5@cloudshell:~/cloudshell_open/visual-commerce-gemini-3-alloydb-1/agents/vision-agent (code-vipassana-test)$ curl http://localhost:8081/.well-known/agent-card.json
{"capabilities":{"streaming":false},"defaultInputModes":["text","image"],"defaultOutputModes":["text"],"description":"Autonomous computer vision agent that audits physical inventory from image payloads using Gemini 3 Flash.", "name":"Vision Inspection Agent", "preferredTransport":"JSONRPC", "protocolVersion":"0.3.0", "skills":[{"description":"Analyzes an image to count and identify inventory items using Gemini 3 Flash code execution.", "examples":["Count the boxes in this warehouse image.", "Analyze this shelf and report item count."], "id":"audit_inventory", "name":"Audit Inventory via Image", "tags":["vision", "counting", "audit", "supply-chain"]}], "url":"http://localhost:8081/", "version":"1.0.0"}mohitbhimrajka5@cloudshell:~/cloudshell_open/visual-commerce-gemini-3-alloydb-1/agents/vision-agent (code-vipassana-test)$
```

常見錯誤：

- 「No such file or directory」：您不在存放區根目錄中。使用 `cd ../..` 返回存放區根目錄，或使用 `pwd` 檢查位置。
- 通訊埠 8081 正在使用中：終止現有程序：`lsof -ti:8081 | xargs kill -9`
- 未設定 `GEMINI_API_KEY`：使用 `cat .env | grep GEMINI_API_KEY` 檢查 `.env` 檔案是否存在於存放區根目錄中
- API 金鑰無效：確認金鑰正確且未過期
- 找不到 `google-genai`：安裝依附元件：`pip install -r requirements.txt`

7. 步驟 3：握手 (A2A 代理商卡片)

注意：如果您已完成 Cloud Shell 教學課程，就已建立這個代理程式資訊卡。本節說明 A2A 通訊協定，以及為何該通訊協定可實現即插即用的代理程式組合。

我們的代理程式會看到問題 (Vision)，並知道供應商 (Memory)。A2A 通訊協定可實現**動態探索**，前端會讀取每個代理的資訊卡，瞭解如何與代理通訊。

A2A 與傳統 REST API

方面	傳統 REST	A2A 通訊協定
端點探索	設定中硬式編碼的網址	透過 /.well-known/agent-card.json 動態設定
功能說明	API 文件 (供人閱讀)	技能 (機器可讀)
整合	每項服務的手動代碼	語意比對：「我需要搜尋庫存」→ 發現技能
已新增代理人	更新所有用戶端的設定	無須設定，系統會自動探索

實際效益：在傳統微服務中，如果新增第三個「物流代理」，您需要使用其網址和 API 合約更新 Control Tower 的程式碼。透過 A2A，Control Tower 會自動探索代理，並透過自然語言技能說明瞭解代理的功能。

因此，A2A 支援**即插即用代理程式組合**，也就是自主系統的架構模式。

建立代理程式卡片

 **路徑檢查：**如果仍在 agents/vision-agent/ 中，請先使用 cd ../../ 返回存放區根目錄

代理資訊卡已包含在 agents/supplier-agent/agent_card.json 中。開啟並查看：

```
{
  "name": "Acme Supplier Agent",
  "description": "Autonomous fulfillment for industrial parts via AlloyDB ScaNN.",
  "version": "1.0.0",
  "skills": [{
    "id": "search_inventory",
    "name": "Search Inventory",
    "description": "Searches the warehouse database for semantic matches using AlloyDB ScaNN vector search.",
    "tags": ["inventory", "search", "alloydb"],
    "examples": ["Find stock for Industrial Widget X-9", "Who supplies ball bearings?"]
  }]
}
```

您可以視需要自訂名稱、說明或範例，以符合您的用途。

重新啟動供應商代理程式，載入資訊卡：

步驟 1：停止執行中的代理程式

```
pgrep -f "uvicorn main:app.*8082"
```

步驟 2：前往代理目錄

```
cd agents/supplier-agent
```

步驟 3：再次啟動代理程式

```
uvicorn main:app --host 0.0.0.0 --port 8082 > /dev/null 2>&1 &
```

> /dev/null 2>&1 & 會在背景執行伺服器並抑制輸出內容，以免中斷終端機。

步驟 4：驗證新的代理程式資訊卡 (啟動後等待 2 到 3 秒)

```
curl http://localhost:8082/.well-known/agent-card.json
```

預期輸出內容：JSON，其中包含您填入的名稱、說明和技能。

```
mohitbhimrajka5@cloudshell:~/cloudshell_open/visual-commerce-gemini-3-alloydb-1/agents/supplier-agent (code-vipassana-test)$ curl http://localhost:8082/.well-known/agent-card.json
{"capabilities":{"streaming":false},"defaultInputModes":["text","application/json"],"defaultOutputModes":["text","application/json"],"description":"Autonomous fulfillment for industrial parts via AlloyDB ScaNN.", "name":"Acme Supplier Agent", "preferredTransport":"JSONRPC", "protocolVersion":"0.3.0", "skills":[{"description":"Searches the warehouse database for semantic matches using AlloyDB ScaNN vector search.", "examples":["Find stock for Industrial Widget X-9", "Who supplies ball bearings?"]}, {"id":"search_inventory", "name":"Search Inventory", "tags":["inventory", "search", "alloydb"]}]}1, "url":"http://localhost:8082/", "version":"1.0.0"}mohitbhimrajka5@cloudshell:~/cloudshell_open/visual-commerce-gemini-3-alloydb-1/agents/supplier-agent (code-vipassana-test)$
```

8. 步驟 4：控制中心

注意：如果您已完成 Cloud Shell 教學課程，服務應該已在執行中。跳至下方的「測試系統」一節，存取 Control Tower 並觀看自主工作流程的運作情形。

使用 FastAPI + WebSockets 執行 Control Tower 前端。這項服務會透過 A2A 探索代理程式，並協調整個迴圈，即時更新。

啟動所有服務

啟動所有服務最簡單的方法：

確認您位於存放區根目錄

```
pwd # Should end with: visual-commerce-gemini-3-alloydb
```

接著：

```
sh run.sh
```

這個單一指令會啟動：

- 通訊埠 8081 的 Vision Agent
- 通訊埠 8082 上的供應商代理程式
- Control Tower on port 8080

等待約 10 秒，讓所有服務完成初始化。

 **疑難排解提示：**如果 `run.sh` 失敗並顯示「**address already in use**」錯誤：

停止所有服務，然後再試一次

```
pkill -f uvicorn
```

```
sh run.sh
```

如果服務仍在執行先前的嘗試，就會發生這種情況。

測試系統

存取 Control Tower：

1. 按一下 Cloud Shell 工具列中的「網頁預覽」按鈕 (眼睛圖示 )
2. 選取「透過以下通訊埠預覽：8080」
3. 系統會在新的分頁中開啟 Control Tower 資訊主頁

注意：前端會使用即時 WebSocket，提供真人服務專員的最新資訊。您會看到代理程式卡片在思考、分析及採取行動時更新，不需要重新整理頁面。

執行示範：

1. 右上角：連線狀態 (綠色的「Live」點)、DEMO/AUTO 模式切換開關和音訊控制選項
2. 中心：主要工作流程畫布，可上傳圖片及查看分析結果
3. 側邊面板 (分析期間會顯示)：工作流程時間軸 (左側)、進度追蹤和程式碼檢視器 (右側)

方法 1：快速入門 (建議)

1. 首頁會顯示「快速入門」部分，並提供範例圖片
2. 按一下任一範例圖片，即可自動開始分析
3. 觀看自主工作流程 (約 30 至 45 秒)

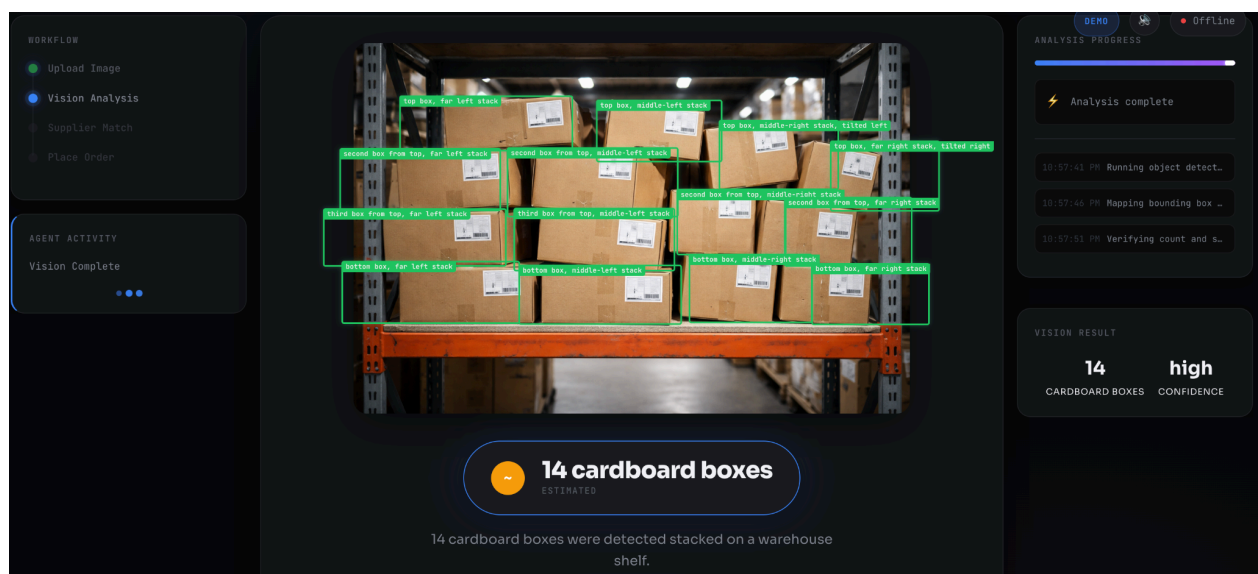
選項 2：上傳自己的檔案

1. 拖曳或點選倉庫/貨架圖片 (PNG、JPG，大小上限為 10 MB) 即可瀏覽
2. 按一下「Initiate Autonomous Workflow」(啟動自主工作流程)
3. 觀察 4 階段管道

發生情況：

1. 代理探索：A2A 通訊協定強制回應會顯示 Vision 代理和供應商代理資訊卡，以及相關技能和端點
2. 影像分析：Gemini 3 Flash 會生成並執行 Python 程式碼 (OpenCV)，計算項目數量。進度列會顯示子步驟。在偵測到的項目上疊加定界框。結果徽章顯示「✓ 程式碼已驗證」或「~ 預估」
3. 供應商比對：AlloyDB ScaNN 向量搜尋動畫。顯示搜尋查詢 (例如「工業用金屬盒」)。結果資訊卡會顯示相符零件、供應商和可信度分數
4. 已下單：收據卡片，內含訂單 ID、數量和詳細資料

提示：請開啟「DEMO」模式 (位於右上方)，以便在簡報期間暫停每個階段。在 AUTO 模式下，工作流程會持續執行。



發生什麼事

控制塔使用 A2A 通訊協定，透過 `/.well-known/agent-card.json` 探索兩個代理程式，協調視覺分析 (Gemini 3 Flash，可執行程式碼)、執行向量搜尋 (AlloyDB ScaNN)，並下達自主訂單，所有作業都透過 WebSocket 即時更新。每個代理都會透過 A2A 標準公開自身功能，因此無需自訂 SDK 即可進行隨插即用的組合。瞭解詳情：[A2A 通訊協定](#)

疑難排解

路徑相關錯誤：

- 執行指令時顯示「找不到你所指定的檔案或目錄」：你不在存放區根目錄中。

```
# Check where you are
pwd

# If you're lost, navigate to home and back to repo
cd
cd visual-commerce-gemini-3-alloydb
```

服務錯誤：

- 「**Address already in use**」：先前執行的程序仍處於有效狀態。

```
# Kill all services and restart
pkill -f uvicorn
sh run.sh # Or manually restart individual agents
```

- 服務未啟動：檢查通訊埠是否遭佔用：

```
# Check which processes are using the ports
lsof -i :8080 # Control Tower
lsof -i :8081 # Vision Agent
lsof -i :8082 # Supplier Agent
```

- AlloyDB 拒絕連線**：確認 AlloyDB 執行個體已啟用公開 IP
-

步驟 9 · 約 0 分鐘

9. 📁 額外內容：部署至 Cloud Run

選用 - 一切在本機運作！但如要透過公開網址分享創作內容，請按照下列步驟操作：

```
# From repo root
sh deploy/deploy.sh
```

會發生什麼情況：

1. 讀取 .env 設定
2. 要求提供您的姓名 (顯示在已部署的應用程式中)
3. 將所有 3 項服務部署為單一 Cloud Run 容器
4. 授予 AlloyDB 存取權的 IAM 角色
5. 輸出可分享的網址

訪客開啟網址後，會看到彈出式視窗：

🚀 由「*你的名字*」部署 - 採用 Gemini 3 Flash、AlloyDB AI 和 A2A 通訊協定

已完成第 14 季「程式碼內觀」

[親自試用程式碼研究室 →]

步驟 10 · 約 0 分鐘

10. 清除所用資源

為避免產生費用，請使用自動清除指令碼終止所有資源：

```
# From repo root
sh deploy/cleanup.sh
```

這項操作會安全移除：

- AlloyDB 叢集 (主要費用驅動因素)
- Cloud Run 服務 (如已部署)
- 相關聯的服務帳戶

指令碼會先要求確認，再刪除任何內容。

11. 參考資料與延伸閱讀資源

本程式碼研究室中的所有技術聲明，都經過 Google Cloud 和 Google AI 官方文件驗證。

官方說明文件

Gemini 3 Flash：

- 程式碼執行 API：<https://cloud.google.com/vertex-ai/generative-ai/docs/model-reference/code-execution-api>
- 開發人員指南：<https://ai.google.dev/gemini-api/docs/gemini-3>
- 模型說明文件：<https://docs.cloud.google.com/vertex-ai/generative-ai/docs/models/gemini/3-flash>
- 模型資訊卡：<https://deepmind.google/models/gemini/flash/>

AlloyDB AI 和 ScaNN：

- ScaNN 效能基準：<https://cloud.google.com/blog/products/databases/how-scann-for-alloydb-vector-search-compares-to-pgvector-hnsw>
- 瞭解 ScaNN 索引：<https://cloud.google.com/blog/products/databases/understanding-the-scann-index-in-alloydb>
- AlloyDB AI 深入探討：<https://cloud.google.com/blog/products/databases/alloydb-ais-scann-index-improves-search-on-all-kinds-of-data>
- 最佳調整做法：<https://docs.cloud.google.com/alloydb/docs/ai/best-practices-tuning-scann>
- AlloyDB 說明文件：<https://cloud.google.com/alloydb/docs>

MCP Toolbox for Databases (替代方法)：

- MCP Toolbox：<https://mcp-toolbox.dev/documentation/introduction/>

價格資訊：

- Gemini API 價格：<https://ai.google.dev/gemini-api/docs/pricing>
- AlloyDB 定價：<https://cloud.google.com/alloydb/pricing>
- Vertex AI 定價：<https://cloud.google.com/vertex-ai/pricing>

已驗證的效能聲明

功能	著作權聲明	資料來源
ScaNN 與 HNSW (已篩選) 的比較	速度快了 10 倍	Google Cloud 網誌 (已驗證)
ScaNN 與 HNSW (標準)	速度快了 4 倍	Google Cloud 網誌 (已驗證)
ScaNN 記憶體用量	縮小 3 到 4 倍	Google Cloud 網誌 (已驗證)
ScaNN 索引建構時間	速度快了 8 倍	Google Cloud 網誌 (已驗證)
程式碼執行逾時	最多 30 秒	Google Cloud 文件 (已驗證)

程式碼執行檔案 I/O	不支援	Google Cloud 文件 (已驗證)
Temperature=0 行為	確定性輸出	經社群驗證

其他資源

Agent2Agent (A2A) 通訊協定：

- A2A 可讓代理探索和通訊標準化
- 在 `/.well-known/agent-card.json` 顯示的代理資訊卡
- 自主代理程式協作的新興標準

ScaNN 研究：

- 根據 Google 研究 12 年的成果
 - 為 Google 搜尋和 YouTube 提供數十億規模的支援
 - 正式發布日期：2024 年 10 月
 - 第一個 PostgreSQL 向量索引，適用於數百萬到數十億個向量
-

12. 挑戰模式：提升代理能力

您已建構出可運作的自主供應鏈。準備好進一步提升收益了嗎？這些挑戰會將您學到的模式應用於新問題。

挑戰 1：以圖片為依據的搜尋 (多模態嵌入)

目前流程：Vision Agent 計算項目 → 生成文字查詢 → Supplier Agent 嵌入文字 → 搜尋 AlloyDB

挑戰：完全略過文字，直接將裁剪後的圖片傳送給供應商專員。

提示：

1. Vision Agent 的程式碼執行作業可以從架子圖片裁剪個別商品
2. Vertex AI 的 `multimodalembembedding@001` 模型可直接嵌入圖片
3. 修改 `inventory.py`，接受圖片位元組而非文字
4. 更新 A2A 技能說明，指出「接受：image/jpeg 或 text」

重要性：對於外觀複雜的零件 (顏色差異、損壞、包裝差異)，影像搜尋的準確度更高。

挑戰 2：可觀測性 - 公開相關資訊來建立信任

目前狀態：系統運作正常，但您無法查看「幕後」情況

挑戰：檢查 AlloyDB 的查詢記錄，證明向量搜尋正在執行。

步驟：

- AlloyDB 預設會啟用查詢洞察功能。如要驗證，請執行：

```
gcloud alloydb instances describe INSTANCE_NAME \
  --cluster=CLUSTER_NAME \
  --region=us-central1 \
  --format="value(queryInsightsConfig.queryPlansPerMinute)"
```

- 透過使用者介面執行供應商搜尋
- 查看實際執行的 SQL：

```
gcloud logging read \
  'resource.type="alloydb.googleapis.com/Instance" AND textPayload:"ORDER BY part_embedding"' \
  --limit 5 \
  --format=json
```

預期輸出內容：您會看到確切的 `ORDER BY part_embedding <=> $1::vector LIMIT 1` 查詢，以及執行時間。

重要性：可觀測性有助於建立信任感。當利害關係人詢問「這個代理程式如何做出決策？」時，您可以向他們展示查詢計畫，而不只是輸出內容。

挑戰 3：多代理程式組合

挑戰：新增第三個代理程式 (物流代理程式)，根據倉庫位置和商品重量計算運費。

架構：

- Vision Agent 輸出內容：項目數量
- 供應商代理程式輸出內容：供應商位置
- 物流代理人 (新) 輸入：目的地、重量 → 輸出：運費 + 預計送達時間

提示：A2A 通訊協定可輕鬆完成這項作業，只要建立含有 `calculate_shipping` 技能的新代理程式資訊卡即可。Control Tower 會自動探索該帳戶。

您要學習的模式：這是以代理為導向的架構核心，也就是由小型可組合的專家建構複雜系統。

13. 結論

您已成功從「生成式 AI」切換至「代理式 AI」。

我們建構的內容：

- **Vision**：我們已將「猜測」取代為**程式碼執行** (透過 API 金鑰使用 Gemini 3 Flash)。
- **記憶**：我們已將「緩慢搜尋」替換為 **AlloyDB ScaNN** (透過 GCP)。
- **行動**：我們已將「API 整合」替換為 **A2A 通訊協定**。

混合式架構的優點：

本程式碼研究室示範了混合式做法：

- **Vision Agent**：使用 Gemini API (API 金鑰) - 提供簡單的免費層級，不需支付 Google Cloud 費用
- **供應商代理**：使用 GCP (Vertex AI + AlloyDB) - 企業級，符合法規

這是自主經濟的架構。你可以保留這組代碼。

後續步驟

- [取得 Gemini API 金鑰](#) (提供免費層級)
 - [查看 Gemini API 說明文件](#)
 - [查看 AlloyDB AI 說明文件](#)
 - [探索 Gemini 程式碼執行功能](#)
-